

HASHDIT

HashDit

Smart Contract Security

Venus Liquidity Hub

Security Audit Report

Venus

26 Jun 2026 — 7 July 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	26 Jun 2026 — 7 July 2026
Overall Risk [Optional]	**Medium**
Project Name	Venus
Github	https://github.com/VenusProtocol/venus-liquidity-hub/pull/2 Fix: https://github.com/VenusProtocol/venus-liquidity-hub/pull/4
Commit	feat/VPD-1300

Smart Contract List

No.	Contract Name	Verdict	Details
1	Hub.sol	Medium	[M01]-Ack [L02]-Ack [L04]-Fixed [I02]-Ack [I03]-Ack [I04]-Fixed [I05]-Fixed
2	HubStorage.sol	Green	-
3	HubAdminLib.sol	Low	[L01]-Fixed
4	YieldGroup.sol	Green	-
5	YieldGroupFRV.sol	Low	[L02]-Ack
6	YieldGroupBase.sol	Medium	[M01]-Ack [L01]-Fixed [L03]-Fixed [L04]-Fixed [I01]-Fixed
7	YieldGroupBaseStorage.sol	Green	-
8	AdapterCoreV1.sol	Green	-
9	AdapterFlux.sol	Green	-
10	AdapterFRV.sol	Medium	[M01]-Ack [L02]-Ack [L05]-Ack
11	Migrator.sol	Green	-

Findings

[M01] FRV positions are marked above realizable value with no anti-sandwich lever active at launch, letting an informed LP exit a coming shortfall onto remaining LPs	
Contract	AdapterFRV.sol, Hub.sol, YieldGroupBase.sol
Severity	Medium

Description	A drawn FRV loan is valued on a straight line as it accrues coupon, and that mark only steps DOWN to realizable value at on-chain settlement.
	
	AdapterFRV._accruedTotalAssets returns rt.totalRaised + (netInterest * elapsed) / lockDuration during Lock, and holds the FULL coupon flat (elapsed = lockDuration) through PendingSettlement / SettlementDeadlineExceeded:
	
	JavaScript
	`// AdapterFRV — _accruedTotalAssets`
	`function _accruedTotalAssets(IFRVVault vault, IFRVVault.FRVVaultRuntime memory rt) internal view returns (uint256) {`
	`if (!rt.fundsWithdrawn) return rt.totalRaised;`
	
	`bool isLock = rt.state == IFRVVault.FRVVaultState.Lock;`
	`bool isPostTerm = rt.state == IFRVVault.FRVVaultState.PendingSettlement || rt.state == IFRVVault.FRVVaultState.SettlementDeadlineExceeded;`
	`if (!isLock && !isPostTerm) return rt.totalRaised;`
	`...`
	`uint256 netInterest = totalInterest - (totalInterest * cfg.reserveFactor) / MANTISSA_ONE;`
	`uint256 elapsed = lockDuration;`
	`if (isLock) {`
	`uint256 raw = block.timestamp > rt.lockStartTime ? block.timestamp - rt.lockStartTime : 0;`
	`elapsed = raw < lockDuration ? raw : lockDuration;`
	`}`
	`return rt.totalRaised + (netInterest * elapsed) / lockDuration;`
	`}`
	
	`// AdapterFRV — maxWithdraw`
	`function maxWithdraw(address resource, address holder) external view override returns (uint256) {`
	`return IFRVVault(resource).maxWithdraw(holder); // returns 0 during Lock`
	`}`
	
	The mark only steps DOWN to realizable value on the transition into a terminal state, and the transition into Liquidated is not time-based — it requires an explicit liquidate / repayBadDebt call (InstitutionalLoanVault.sol:203-224, 276-294). So there is a public, on-chain-observable window where a loan is already known to be defaulting — state == SettlementDeadlineExceeded with outstandingDebt > 0 (institution missed the deadline), or collateral value below debt (liquidatable) — yet _accruedTotalAssets still

	marks it at totalRaised + full netInterest. FRV itself has maxWithdraw = 0 during this window, but the Hub's PPS is a weighted result across all YieldGroups, so any LP can read this signal and redeem before the settlement tx through Core/Flux's liquidity; it is a permissionless race, not privileged information. **Impact**: Direct effect — a zero-sum transfer of the FRV loss from informed exiters to remaining LPs. When a drawn FRV loan settles below its marked value (any institution default / partial repayment / bad-debt liquidation), the loss is not shared pro-rata. An LP who redeems during the observable pre-settlement window extracts full marked value through the liquid sibling YieldGroups, and the entire realized shortfall lands on whoever stays. In the worked example: attacker +238,095, stayers −238,095 on a single 1,000,000 loan with a 40% shortfall.
Recommendation	Do not launch an FRV-bearing Hub with the mark left forward-looking and no friction. The three levers are not equally effective (verified against the real InstitutionalLoanVault state machine): (a) — root cause, primary: mark the position down at the first objectively observable default signal instead of waiting for the terminal settlement event. Today _accruedTotalAssets holds the mark at totalRaised + full netInterest all the way through SettlementDeadlineExceeded, and the transition into a terminal state (Liquidated/Failed) is not time-based — it requires someone to call liquidate/repayBadDebt. The mark should instead step down to a conservative floor (e.g. principal only, or principal net of any known collateral shortfall) the moment state == SettlementDeadlineExceeded && outstandingDebt > 0 — i.e. the repayment deadline has passed with debt still outstanding, a condition that is already publicly observable on-chain. PendingSettlement (freshly matured, still inside the grace period) does not need to change. If the institution later repays in full, the mark can be raised back to full value at Matured — correcting upward creates no sandwich risk, since nobody is harmed by having bought in earlier at a lower, later-corrected price. This is the only lever that closes the observable window itself rather than merely taxing it. (b) — backstop, secondary: a non-zero retained redeemFeeBps taxes the exit and hands it to stayers, but it is a flat tax on exit size while the harm scales with shortfall × size; fully neutralizing the example would need a ~24% fee, so a realistic fee (≥ 100 BPS recommended at launch) only dampens the extraction and penalizes honest exiters along with opportunistic ones. Use it as a safety margin on top of (a), not as a substitute for it. (c) — exposure control, tertiary: keep the per-YieldGroup FRV cap conservative, and until (a) ships, consider applying a manual haircut to the FRV valuation input so the marked value already prices in some probability of shortfall instead of assuming full repayment (note: the README's coupon-haircut lever, setResourceHaircutBps, was removed from the code — see the README-drift finding).
Status	Acknowledged. This is a known, accepted design gap. We explored an alternative (auto-marking the position down at SettlementDeadlineExceeded && outstandingDebt > 0) but verified against the real vault state machine that this transition can resolve to

	Matured (late-but-fully-repaid), reproducing the same upward-jump arbitrage the team's reverted coupon-haircut (7b02aca) already ran into — no clean on-chain valuation fix exists without a continuously-updated external default-probability input. Our one incremental suggestion: escalate redeemFeeBps automatically on entry into SettlementDeadlineExceeded rather than relying on manual governance arming, closing the reaction-lag gap in the existing cap + exit-fee mitigation. Not a substitute for it — the team's current bounded-risk approach (per-source cap + optional exit fee) stands as the accepted mitigation.

[L01] YieldGroup deregistration can be permanently griefed by a 1-wei asset donation	
Contract	HubAdminLib.sol, YieldGroupBase.sol
Severity	Low

Description	Hub.removeYieldGroup refuses to deregister a YieldGroup while it reports any custodied value:
	
	JavaScript
	`// HubAdminLib.removeYieldGroup`
	`try IYieldGroupBase(yieldGroup).totalAssets() returns (uint256 v) {`
	`  balanceReadable = true;`
	`  yieldGroupBal = v;`
	`} catch {`
	`  balanceReadable = false;`
	`}`
	`if (balanceReadable && yieldGroupBal != 0) revert IHub.YieldGroupHasBalance(yieldGroup, yieldGroupBal);`
	
	But YieldGroupBase.totalAssets() adds the contract's raw asset-token balance on top of resource-backed value:
	
	JavaScript
	`function totalAssets() external view virtual override returns (uint256 total) {`
	`  for (uint256 i; i < len; ++i) {`
	`  total += IResourceAdapter(_resourcesConfig[resource].adapter).totalAssets(resource, address(this));`
	`  }`
	`  total += IERC20(_assetToken).balanceOf(address(this)); // <-- idle/donated balance counted`
	`}`
	
	Anyone can transfer 1 wei of the asset token directly to a drained YieldGroup. totalAssets() then returns ≥ 1, so removeYieldGroup reverts with

YieldGroupHasBalance. Because totalAssets() returns normally (it doesn't revert), the emergency-eviction branch — which only triggers when the view reverts — does not apply.

Vulnerable code

```javascript
// contracts/YieldGroup/base/YieldGroupBase.sol:403-412
function totalAssets() external view virtual override returns (uint256
total) {
    ...
    total += IERC20(_assetToken).balanceOf(address(this)); // dust
donation lands here
}

// contracts/Hub/lib/HubAdminLib.sol:97
if (balanceReadable && yieldGroupBal != 0) revert
IHub.YieldGroupHasBalance(yieldGroup, yieldGroupBal);
```

Impact

- An attacker can block deregistration of any healthy YieldGroup for ~1 wei + gas. removeYieldGroup is VIP/timelock-gated, so the queued proposal is visible in advance; the attacker front-runs each execution with a fresh 1-wei donation and the removal reverts every time.
- The donated dust is technically drainable (withdraw consumes idle first, YieldGroupBase.sol:268), but removeYieldGroup itself does not drain, and reallocate-then-remove are separate transactions — so there is no atomic drain+remove that can't be re-griefed between txs.
- No funds are lost; the damage is operational — governance cannot cleanly retire an unwanted YieldGroup, leaving a stale registry/queue entry.

Recommendation

When gating removeYieldGroup on YieldGroupBase.totalAssets(), net out the YieldGroup's idle asset-token balance (IERC20(asset()).balanceOf(yieldGroup)) from the check — only resource-backed value should block removal. totalAssets() itself should remain unchanged (idle stays counted in NAV/PPS for LPs); the netting applies only to the removal-eligibility check.

Status

Fixed in commit f170af0. removeYieldGroup now nets the YieldGroup's idle asset balance out of the totalAssets() gate, so only genuine resource-backed value blocks removal:

```javascript
  try IYieldGroupBase(yieldGroup).totalAssets() returns (uint256 v) {
      balanceReadable = true;
      uint256 idle = IERC20(hubAsset).balanceOf(yieldGroup);
```

```
            yieldGroupBal = v > idle ? v - idle : 0;
        } catch {
            balanceReadable = false;
        }
```

[L02] A defaulted FRV vault pays lender compensation in the collateral token, which the Hub's single-asset accounting strands on the Hub instead of distributing to LPs	
Contract	AdapterFRV.sol, YieldGroupFRV.sol, Hub.sol
Severity	Low

Description

The Hub/YieldGroup/adapter model an FRV vault as a pure single-supply-asset ERC-4626 (read/settle only in asset() units), but the real InstitutionalLoanVault pays a second token — confiscated collateral margin — to the withdrawal receiver on an institution-default settlement, via an ERC-4626 _afterWithdrawHook the adapter neither knows about nor accounts for.

When fundraising raises >= minBorrowCap but the institution has not posted full collateral by openEndTime, the vault enters Failed (Scenario B) and confiscates the institution's margin for lender compensation: InstitutionalLoanVault.sol:593-605 — _enterFailed sets institutionDefaulted = true; confiscatedMarginRemaining = marginAmount. Thereafter, every supplier withdrawal distributes that margin pro-rata in the collateral token:

JavaScript

```
//InstitutionalLoanVault.sol:621-636 — _afterWithdrawHook

function _afterWithdrawHook(
        address receiver,
        uint256 shares
    ) internal override {
        if (!_instRuntime.institutionDefaulted ||
_instRuntime.confiscatedMarginRemaining == 0) return;

        // totalSupply() is post-burn; reconstruct pre-burn total for
pro-rata calculation
        uint256 totalSharesBeforeBurn = totalSupply() + shares;
        uint256 compensation = (_instRuntime.confiscatedMarginRemaining
* shares) / totalSharesBeforeBurn;

        if (compensation > 0) {
            _instRuntime.confiscatedMarginRemaining -= compensation;
```

```
            _releaseCollateral(receiver, compensation); //@audit
collateral release here
            emit MarginCompensationClaimed(receiver, compensation);
        }
    }
```

The adapter's withdraw passes the Hub as receiver:

```javascript
//AdapterFRV.sol:99-101

IFRVVault(resource).withdraw(amount, to,
address(this)),

//and every Hub withdraw/reallocate path sets to == address(this) (the
Hub) — Hub.sol:1333

IYieldGroupBase(yieldGroup).withdraw(toPull, address(this)), and
Hub._reallocatePullAll.
```

So on a Scenario-B FRV, the vault delivers the supply-asset principal to the Hub and the collateral-token margin compensation to the Hub. The Hub then forwards only the supply asset to the LP and its accounting only ever sees the asset:

```javascript
totalAssets() = Σ YieldGroup.totalAssets() + asset().balanceOf(Hub)
(Hub.sol:774-781)
```

never counts the collateral token, and the withdraw-leg delta check measures only _assetToken , so the foreign token passes silently.

The FRV NAV in Failed is **previewRedeem(settlementAmount) == totalRaised (principal only; AdapterFRV.sol:208-213)**, so LP shares reflect principal alone — the margin compensation, which is precisely the lenders' remedy for the default, accrues to the Hub as an unaccounted, non-distributed collateral balance.

Impact:

the default-compensation owed to Hub LPs does not reach them through share price; it strands on the Hub and is recoverable only by governance sweep(collateralToken, to) to a treasury address, not pro-rata to the LPs who bore the loss, so absent manual off-protocol redistribution the entitled value is effectively mis-routed. Bounded to Low

	because no protocol funds are lost (the collateral is sweep-recoverable) and it requires registering a collateralized InstitutionalLoanVault (collateral token != supply asset) that then hits the Scenario-B default; if collateralAsset == asset() the compensation would instead land as Hub idle and flow to LPs via PPS, so the behavior is silently config-dependent. Distinct from M-1 (which is the NAV over-mark on a drawn, Lock-state loan); this is the Failed-fundraising collateral side channel.
Recommendation	Decide explicitly how the FRV family handles the collateral-compensation side channel and enforce it, rather than leaving it silent. Options: (a) at registration (AdapterFRV.validateRegistration / YieldGroupFRV), reject any FRV vault whose collateral asset differs from the Hub asset, or whose _afterWithdrawHook can pay a foreign token, so the Hub only ever holds vaults it can fully account; (b) have the adapter/YieldGroup detect and value the collateral balance received on withdraw (price it and fold it into NAV, or convert it to the supply asset) so the compensation flows to LPs via PPS; or (c) if relying on governance sweep, document a runbook that sweeps the collateral and redistributes it pro-rata to the affected LP set, and monitor MarginConfiscated / MarginCompensationClaimed events. At minimum, add this to the FRV watch-list and the README so operators know a defaulted collateralized FRV leaves collateral dust on the Hub.
Status	Acknowledged. Root cause (collateral-token compensation stranding on the Hub) is unchanged in code; the team adopted the off-chain runbook option from our own recommendation set (sweep the collateral, sell it off-chain, donate proceeds back to raise PPS) rather than an on-chain fix, and documented the behavior in commit 909afe2.

[L03] YieldGroup Lacks Non-Asset Token Recovery Mechanism	
Contract	YieldGroupBase.sol
Severity	Low
Description	The Hub contract provides a governance-gated sweep function to recover accidentally sent non-asset ERC-20 tokens. The YieldGroup layer has no equivalent. Any non-asset token (XVS rewards, airdrops, mistakenly transferred ERC-20s) that enters a YG contract address is permanently locked with no recovery path.

JavaScript

```
  // YieldGroupBase — totalAssets (no sweep function; Hub has one)
  function totalAssets() external view virtual override returns (uint256
total) {
      address[] storage list = _resources;
      uint256 len = list.length;
      for (uint256 i; i < len; ++i) {
          address resource = list[i];
          total += IResourceAdapter(_resourcesConfig[resource].adapter)
              .totalAssets(resource, address(this));
      }
      total += IERC20(_assetToken).balanceOf(address(this));
```

```
        // No sweep function: non-assetToken ERC-20s sent here are
permanently locked
    }
```

Impact:
Current BSC mainnet state: venusSupplySpeeds[vUSDT] = venusSupplySpeeds[vUSDC] = 0 — no XVS distribution active, so there is no immediate impact. If Venus re-enables rewards for registered vToken markets in the future, all accumulated XVS will be permanently inaccessible in YG contracts. Airdrops and mistransferred tokens face the same fate.

Recommendation

Add a governance-gated sweep function to YieldGroupBase, explicitly barring recovery of _assetToken:

```javascript
    function sweep(address token, address to) external {
        _checkAccessAllowed("sweep(address,address)");
        require(token != _assetToken, "Cannot sweep asset");
        IERC20(token).safeTransfer(to,
IERC20(token).balanceOf(address(this)));
    }
```

Status

Fixed in commit [25b1ea7](). Added a governance-gated sweep to YieldGroupBase, barring both asset() and every registered resource (the receipt token backing NAV in every current family):

```javascript
    function sweep(address token, address to) external override
nonReentrant {
        _checkAccessAllowed("sweep(address,address)");
        if (to == address(0)) revert ZeroAddress();
        if (token == _assetToken) revert SweepProtectedAsset();
        if (_resourcesConfig[token].registered) revert
SweepProtectedResource(token);
        ...
    }
```

[L04] reallocate Can Push Funds Into YGs or Resources Not Covered by Withdraw Queues	
Contract	Hub.sol, YieldGroupBase.sol
Severity	Low
Description	_reallocatePushOne only validates that the target YG is registered and unpaused, without verifying its presence in _outerWithdrawQueue. depositResource similarly validates registration and pause state only, without checking _innerWithdrawQueue membership. Funds pushed to a non-queue target are counted in totalAssets()

	(affecting PPS), but _routeWithdraw / YG.withdraw() cascades do not traverse that target, making the funds unreachable through normal withdrawal paths.
	
	JavaScript
	`// Hub – _reallocatePushOne (key portion)`
	`IHub.YieldGroupConfig storage cfg = _yieldGroups[depositLeg.yieldGroup];`
	`if (!cfg.registered) revert YieldGroupNotRegistered(depositLeg.yieldGroup);`
	`if (cfg.paused) revert YieldGroupIsPaused(depositLeg.yieldGroup);`
	
	`// ← No check: is depositLeg.yieldGroup in _outerWithdrawQueue?`
	`// YieldGroupBase – depositResource (key portion)`
	`if (!cfg.registered) revert ResourceNotRegistered(resource);`
	`if (cfg.paused) revert ResourceIsPaused(resource);`
	`// ← No check: is resource in _innerWithdrawQueue?`
	
	Impact:
	If the Operator pushes funds into a non-queue target and other YGs/resources exhaust their liquidity, user withdrawals will fail. Funds are not permanently lost — updating the queues (which coverage checks will force to include any funded targets) restores access — but users cannot fully withdraw until the Operator corrects the configuration.

Recommendation	Add a queue membership check in _reallocatePushOne before depositing (or trigger the coverage check immediately after); or explicitly document in operator runbooks that the withdraw queue must be updated after any reallocate push operation.
Status	Fixed. Root cause (no queue-membership check in _reallocatePushOne/depositResource) is unchanged in code; the team chose the documentation alternative from our recommendation over an on-chain guard, treating it as a fully-recoverable operator misconfiguration. Documented in commit c3287fb.

[L05] AdapterFRV.validateRegistration does not enforce the vault's decimalsOffset()==0 assumption	
Contract	AdapterFRV.sol
Severity	Low

Description	All of AdapterFRV's valuation math (_breakdown, _accruedTotalAssets, previewRedeem, etc.) is built on the assumption that the FRV vault's share-to-asset conversion is a standard 1:1 ERC-4626 basis with no decimals offset. The contract's own NatSpec explicitly states this assumption, but validateRegistration is a no-op — any vault passes registration unconditionally, and the assumption is never enforced in code:
	
	JavaScript
	`// AdapterFRV – validateRegistration`
	`/**`
	`* @dev ... vault leaves `_decimalsOffset()` at 0 (it does, and defines no override). If a future FRV`
	`* family overrides it, this adapter must be updated in lockstep.`

```
   */
  function validateRegistration(address resource) external pure override
{
      // No FRV-specific registration precondition. `resource` validity
is established by the
      // YieldGroup's `asset()` probe and `code.length` check at
registration time.
      resource;
  }
```

Impact:
If a future FRV vault is registered with decimalsOffset() != 0 (e.g. a new vault family adding an offset for inflation-attack protection), AdapterFRV's share-to-asset conversions would all be computed incorrectly without reverting — silently producing a wrong NAV that pollutes the Hub's totalAssets/PPS. Verified against the real FRV repository (BaseVault.sol), the current implementation inherits stock OZ ERC4626Upgradeable with no _decimalsOffset() override, so the assumption currently holds and there is no immediately exploitable issue; but this relies entirely on future governance discipline rather than an on-chain guarantee. Scope: only triggers if an incompatible vault is registered in the future — no current risk.

Recommendation

An on-chain require(decimalsOffset() == 0) isn't achievable — OZ's _decimalsOffset() is internal virtual, not part of the ERC-4626 interface, and no FRV vault (current or future) exposes it externally.

1) Primary fix (reliable): a governance checklist item, not a code check. Before registering any new FRV vault, require manual source-review confirmation that its inheritance chain does not override _decimalsOffset(). Document this as a mandatory pre-registration step alongside addResource/updateResourceAdapter.
2) Secondary, best-effort safety net: if the vault is known to be fresh (totalSupply() == 0 && totalAssets() == 0 at registration), OZ's conversion formula reduces to convertToShares(assets) == assets * 10^offset, so a probe check works:

JavaScript

```
  function validateRegistration(address resource) external view override
{
      IFRVVault vault = IFRVVault(resource);
      if (vault.totalSupply() == 0 && vault.totalAssets() == 0) {
          uint256 probe = 1e6;
          require(vault.convertToShares(probe) == probe, "FRV:
unsupported decimalsOffset");
      }
      // totalSupply/totalAssets != 0 → cannot be reliably probed; rely
on the checklist above.
  }
```

	This only covers the zero-state case and cannot substitute for the manual checklist once a vault already has supply/assets at registration time.
Status	Acknowledged. Accepted as a governance-process control rather than a code check, matching our own recommendation: _decimalsOffset() isn't externally readable under the standard ERC-4626 interface, so no reliable on-chain require exists. Governance manually reviews any new FRV vault's inheritance chain before registration.

[I01] README documents a load-bearing ACM-role list and two subsystems that do not match the deployed code	
Contract	README.md, YieldGroupBase.sol
Severity	Informational
Description	The README states the ACM role is keccak256(targetContract, roleString) and that "the asymmetry is entirely a matter of which addresses governance grants each role to" (README:272) — i.e. the role-string list is the operational provisioning spec. **That list (README:282) omits updateResourceAdapter(address,address),** which is the single highest-blast-radius YieldGroup lever: it instantly repoints the delegatecall target for a registered resource (YieldGroupBase.sol:172-190 → :589-592) with no timelock and only code.length/asset-match/view-only validation. **Conversely, the list includes two roles whose functions do not exist anywhere in the code:** setResourceHaircutBps(address,uint16) (the coupon-haircut lever, removed in 7b02aca) and setEmaAlphaBps(uint16). The README further documents an entire APY-EMA subsystem (realizedAPYBps(), refreshAPY, MAX_YIELD_GROUP_APY_BPS, "Hub accrues fees and refreshes APY") and the coupon-haircut valuation lever (README §FRV lifecycle, §Fees) — **a repo-wide search confirms none of haircut, realizedAPYBps, refreshAPY, setEmaAlphaBps, or MAX_YIELD_GROUP_APY_BPS exists in contracts/.** **Impact**: no direct on-chain exploit, but the role list is the source of truth governance uses to provision ACM grants, so the omission leaves the most dangerous lever's tier undocumented while two listed grants are dead; and integrators may build against a realizedAPYBps() / haircut that does not exist. This is operationally load-bearing documentation drift, not a code bug.
Recommendation	Add updateResourceAdapter(address,address) to the README YieldGroup role list with an explicit governance-only tier note (and see the lead on giving it timelock parity with the beacon upgrade). Remove setEmaAlphaBps(uint16) and setResourceHaircutBps(address,uint16) from the role list, and delete (or clearly mark as not-yet-implemented) the APY-EMA and coupon-haircut sections of the README so the spec matches the shipped code. Add a CI check that every _checkAccessAllowed("X") literal has a matching README role entry and vice-versa.
Status	Fixed in commit 92d5005. README ACM role list synced: added updateResourceAdapter and sweep, removed the phantom

	setEmaAlphaBps/setResourceHaircutBps entries, deleted the dead APY-EMA docs section. (CI drift-check between _checkAccessAllowed literals and the README was not added.)

[I02] reallocate does not reject a YieldGroup (or resource) that appears in both the withdraw and deposit legs, wasting gas on a no-op round-trip	
Contract	Hub.sol
Severity	Informational

Description	reallocate / emergencyReallocate execute all withdraw legs, then all deposit legs, and only enforce a global net-zero sum — they never check that a given YieldGroup (or (yieldGroup, resource) pair) is not present on both sides.
	
	JavaScript
	`//Hub.sol:1165-1182 – _reallocate runs _reallocatePullAll(withdraws) then _reallocatePushAll(deposits, taSnapshot) and reverts only if withdrawSum != depositSum:`
	
	`uint256 withdrawSum = _reallocatePullAll(withdraws);`
	`...`
	`uint256 depositSum = _reallocatePushAll(deposits, taSnapshot);`
	
	`if (withdrawSum != depositSum) revert ReallocateImbalanced(withdrawSum, depositSum);`
	
	Neither _reallocatePullAll nor _reallocatePushOne de-duplicates the YieldGroup across the two arrays. If the operator passes the same YieldGroup Y in both withdraws and deposits (e.g. withdraw 500 from Y, deposit 500 into Y), the pull redeems Y's receipt tokens back to the Hub as idle (YieldGroupBase.withdraw → adapter redeemUnderlying/fToken withdraw) and the push immediately re-deposits the same amount into Y (YieldGroupBase.deposit → adapter mint/fToken deposit), leaving Y exactly where it started.
	
	The operation "reallocates" nothing: it is a no-op that still pays for the full redeem+mint round-trip against the external market (Venus/Fluid), plus the Hub's balance-delta bookkeeping.
	
	Impact:
	pure operator gas waste on a self-cancelling round-trip (and needless wear on the external market's redeem/mint path); no fund loss, no cap bypass, no accounting desync. Because reallocate is Operator-gated (semi-trusted), the only party harmed is the caller who submitted the redundant plan.

Recommendation	Reject a leg whose (yieldGroup, resource) appears in both the withdraw and deposit arrays (revert with a dedicated ReallocateSelfLeg error), so a plan that does not change any allocation cannot be submitted. A stronger form (mirrors MetaMorpho's reallocate)

	is to collapse legs per (yieldGroup, resource) and compute the net delta per entry, enforcing that each entry is strictly a net withdraw or a net deposit — this both eliminates the no-op round-trip and makes operator intent explicit and auditable. Purely defensive; can be scoped as a follow-up.
Status	Acknowledged. Working as intended, the team's position is that a same-YieldGroup leg on both sides of a reallocate plan is a no-op round trip that only wastes the Operator's own gas, with no fund loss or cap bypass, so dedup logic wasn't added.

[I03] reallocate deposit legs check only the governance cap, not the YieldGroup's protocol maxDeposit, so an over-capacity queue leg reverts late after wasted work	
Contract	Hub.sol
Severity	Informational

Description	_reallocatePushOne gates a deposit leg on the Hub's governance policy cap (_effectiveCap) but never on the target YieldGroup's actual protocol-level deposit capacity (IYieldGroupBase.maxDeposit() — the sum of its inner resources' room).
	
	JavaScript
	`//Hub.sol:1248-1257 — the only capacity gate is the policy cap; the push then calls the YieldGroup deposit directly:`
	
	`uint256 cap = _effectiveCap(cfg, taSnapshot);`
	`uint256 currentBal = IYieldGroupBase(depositLeg.yieldGroup).totalAssets();`
	`if (currentBal + depositLeg.amount > cap) revert HubCapacityExceeded(...);`
	`...`
	`uint256 deposited = depositLeg.resource == address(0) ? IYieldGroupBase(depositLeg.yieldGroup).deposit(depositLeg.amount) : IYieldGroupBase(depositLeg.yieldGroup).depositResource(depositLeg.resource, depositLeg.amount);`
	`if (deposited != depositLeg.amount) revert YieldGroupUnderfilled(...);`
	
	When the binding constraint is the protocol room and not the governance cap (Venus Core supply cap reached, an FRV vault not in Fundraising, or a per-resource cap), cap passes but the deposit cannot place the full amount. For the queue path (resource == address(0)), YieldGroupBase.deposit pulls the funds in via safeTransferFrom, cascades over the inner deposit queue placing what it can, safeTransfers the unplaced remainder back, and returns deposited < amount — so _reallocatePushOne reverts YieldGroupUnderfilled only after paying for the pull-in transfer, the full inner cascade, and the refund transfer. The strict revert itself is correct and intended (it preserves the net-zero reallocate invariant — a leg must place exactly its stated amount or the whole plan rolls back); the issue is that it fails late and wastefully.

	This is also a capacity-check inconsistency across the four routing paths. Every other path validates its protocol capacity up front, and the deposit/withdraw sides mirror each other: (1) the Hub's ERC-4626 deposit/withdraw both consult aggregate capacity — deposit via the OZ maxDeposit gate (Hub.sol:784-786, "ERC4626: deposit more than max") and withdraw via maxWithdraw; (2) the targeted reallocate pair is symmetric — depositResource pre-checks amount > room → ResourceCapacityExceeded (YieldGroupBase.sol:314-315) and withdrawResource pre-checks amount > liquid (maxWithdraw) → ResourceLiquidityInsufficient (:347-348); (3) the queue withdraw reverts cleanly inside the YieldGroup with the specific ResourceLiquidityInsufficient (:281). Only the queue deposit breaks the pattern: it neither pre-consults maxDeposit() nor reverts internally — it refunds and returns a partial deposited, so the shortfall surfaces as the generic YieldGroupUnderfilled, an error whose actual purpose is catching a misbehaving adapter, not a legitimate capacity shortfall. Put differently: the != amount delta check is a defensive backstop on the pull side (withdraw reverts-on-short, so it rarely fires) but is the primary capacity-shortfall handler on the push side (deposit returns-partial), which is why the push should carry the same explicit maxDeposit pre-check the other three paths already have. Impact: operator gas waste and a late, less-specific error on a mis-sized reallocate/emergencyReallocate plan, plus the maintainability risk of the one path that diverges from the deposit/withdraw capacity-check symmetry; the transaction reverts atomically, so there is no fund loss, no partial state, and no net-zero break. During an emergencyReallocate unwind, an over-sized push leg reverts and the operator must re-size to the target's live maxDeposit and retry — recoverable, operator-controlled, no permanent block.
Recommendation	In _reallocatePushOne, for the queue path (resource == address(0)), pre-check depositLeg.amount <= IYieldGroupBase(depositLeg.yieldGroup).maxDeposit() and revert early with a specific error (e.g. HubCapacityExceeded / a dedicated YieldGroupCapacityExceeded) before the token transfer and cascade, mirroring the fail-fast ResourceCapacityExceeded check the targeted depositResource path already performs. This makes the two push paths symmetric, fails fast (less gas on a bad plan), and returns an error that tells the operator the binding constraint is protocol room rather than the governance cap. Alternatively, document that operators must size each deposit leg within the target's live maxDeposit() (which is only knowable post-pull, so a fail-fast on-chain check is the more robust option).
Status	Acknowledged. Working as intended, the queue deposit's partial-fill-and-refund is required by the outer _routeDeposit cascade, and YieldGroupUnderfilled already reports the shortfall; a maxDeposit() pre-check was judged not worth the added complexity for an atomically-reverting, operator-gated path.

[I04] Pause Window Exempts Management Fee But Not Performance Fee	
Contract	Hub.sol

Severity	Informational

Description	Hub's pause handling treats the two fee types asymmetrically, and the asymmetry reads as deliberate rather than an oversight — though nothing in the code says so explicitly.
	
	unpauseHub() advances _lastFeeAccrualTimestamp by the exact pause duration, fully excluding the paused window from management-fee accrual: management fee is a time-based custody/availability fee, and no service was rendered while the Hub was paused, so none is charged for that window. By contrast, it never updates _highWaterMarkPerShare — the underlying protocols (Venus Core vTokens, Fluid fTokens) keep accruing real yield throughout the pause regardless of the Hub's own pause state, since being paused only blocks user-facing deposit/withdraw, it does not stop the capital already deployed from earning. So when the Hub resumes, the first _accrueFees() call charges the full performance fee on all genuine gains realized during the pause. This is consistent with the protocol's general fee philosophy: performance fee tracks real profit generated, not user availability.
	
	JavaScript
	`// Hub — unpauseHub`
	`uint64 pauseDuration = nowTs - _pauseStart;`
	`_lastFeeAccrualTimestamp += pauseDuration; // Management fee: pause window excluded — time-based, no service = no charge`
	`// _highWaterMarkPerShare NOT updated`
	`// Performance fee: pause-period yield IS included in the first post-resume accrual — consistent`
	`// with "fee tracks real profit, not availability," but currently undocumented as such`
	`_hubPaused = false;`
	
	Impact:
	Users cannot act (deposit/withdraw) while paused, but the yield their capital continues earning from Core/Flux/FRV during that window is still subject to the normal performance-fee cut once the Hub resumes. This is very likely the intended behavior — the pause is a circuit breaker on user operations, not a suspension of the underlying fee model, and waiving profit-sharing on real, externally-generated gains simply because a pause happened to overlap that period would be inconsistent with how performance fee works everywhere else. The risk is purely reputational/documentation: without an explicit statement of intent, a future auditor, governance member, or user could mistake this for a bug, or dispute being charged a performance fee during a window they had no ability to act in.

Recommendation	Document the rationale explicitly, in both NatSpec on unpauseHub() and in user-facing docs: management fee is waived pro-rata for the pause duration because it is an availability/custody fee; performance fee is deliberately NOT waived, because it tracks real yield generated by the underlying protocols, which continues independent of the Hub's pause state. If the team decides on further review that pause-period performance fees should instead be waived, update _highWaterMarkPerShare to the current PPS inside unpauseHub() to match the management-fee treatment.

Status	Fixed in commit 7a2bab7. NatSpec now documents that the performance fee is deliberately not waived across a pause (profit-share on real yield, not an availability charge); the HWM is only re-anchored on a cohort reset, never on a pause.

[I05] withdraw(maxWithdraw) / redeem(maxRedeem) Violates ERC-4626 Executability Semantics	
Contract	Hub.sol
Severity	Informational

Description	Hub's public withdraw()/redeem() overrides call _accrueFees() before delegating to the OZ ERC4626Upgradeable base implementation. The two functions are affected differently, because maxWithdraw is asset-denominated while maxRedeem is share-denominated:
	
	• withdraw(maxWithdraw()): fee-share minting inside _accrueFees() increases totalSupply without adding assets, which lowers PPS. The caller's own share balance is unchanged, but maxWithdraw(owner) = convertToAssets(balanceOf(owner)) (net of exit fee, capped by liquidity/perTxCap) is computed from PPS, so it shrinks in real, meaningful proportion to however much fee accrued — not a rounding artifact. A stale (pre-accrual) maxWithdraw() value submitted as assets will exceed the freshly-recomputed (smaller) ceiling and revert.
	
	• redeem(maxRedeem()): when redeemFeeBps == 0, maxRedeem(owner) is computed as min(convertToShares(maxWithdraw(owner)), balanceOf(owner)). Converting the owner's asset entitlement back to shares with the same (post-accrual) PPS largely self-corrects the dilution — since the owner's own balanceOf(owner) never changes from fee minting (only feeRecipient's balance grows), the round-trip convertToAssets → convertToShares returns approximately balanceOf(owner) again. The only residual effect is a small (typically 1-2 wei) discrepancy from floor-rounding on each leg of the round-trip — not an effect proportional to the size of the accrued fee.
	
	JavaScript
	`// Hub.sol – public withdraw() override (the actual _accrueFees() call site)`
	`function withdraw(uint256 assets, address receiver, address owner)`
	`public`
	`override(ERC4626Upgradeable, IERC4626Upgradeable)`
	`nonReentrant`
	`whenNotHubPaused`
	`returns (uint256)`
	`{`
	`_accrueFees(); // ← mints fee shares here; totalSupply increases, PPS drops, maxWithdraw(owner) shrinks`
	`return super.withdraw(assets, receiver, owner); // OZ base – re-checks maxWithdraw(owner) AFTER accrual`
	`}`
	
	`// Hub.sol – public redeem() override (same pattern)`

```
    function redeem(uint256 shares, address receiver, address owner)
        public
        override(ERC4626Upgradeable, IERC4626Upgradeable)
        nonReentrant
        whenNotHubPaused
        returns (uint256)
    {
        _accrueFees();
        return super.redeem(shares, receiver, owner);
    }
    // OZ ERC4626Upgradeable.withdraw() — library code, not part of this
repo; this is what super.withdraw() resolves to
    function withdraw(uint256 assets, address receiver, address owner)
public virtual override returns (uint256) {
        uint256 maxAssets = maxWithdraw(owner); // ← re-derived fresh,
AFTER Hub's _accrueFees() already ran
        if (assets > maxAssets) {
            revert ERC4626ExceededMaxWithdraw(owner, assets, maxAssets);
// reverts if caller passed the pre-accrual ceiling
        }
        uint256 shares = previewWithdraw(assets);
        _withdraw(_msgSender(), receiver, owner, assets, shares); //
Hub's internal _withdraw override — runs only after the check passes
        return shares;
    }
```

Note: Hub's own internal _withdraw() override (the _burn/_routeWithdraw/transfer logic) plays no part in this issue — it executes only after the OZ ceiling check above has already passed, and never calls _accrueFees() itself.

Concrete counterexamples:

JavaScript

```
    - withdraw path — meaningful, fee-proportional shrink:
        Pre-tx: maxWithdraw(owner) = 100,000 (queried at PPS = 1.05)
        A large performance-fee accrual dilutes PPS to 1.04 within the
same call
        Post fee-accrual: maxWithdraw(owner) ≈ 99,048
        require(100,000 ≤ 99,048) → REVERT (gap scales with the fee
accrued, not just 1 unit)

    - redeem path — self-correcting, rounding-only shrink:
        Pre-tx: maxRedeem(owner) = 1000 (== balanceOf(owner), not
liquidity-bound)
        Post fee-accrual: maxRedeem(owner) = 999 (balanceOf(owner) is
still 1000; the 1-unit gap is floor-rounding in the convertToAssets →
convertToShares round-trip, independent of how large the fee was)
        require(1000 ≤ 999) → REVERT, but only ever by a unit or two
```

	Impact: withdraw(maxWithdraw()) can revert by a materially large margin when a large fee accrues in the same block (e.g. after a long-idle Hub's first interaction, or a large performance-fee crystallization) — integrators relying on a fresh maxWithdraw() query need to account for real staleness, not just a rounding buffer. redeem(maxRedeem()) can also revert, but the gap is bounded to ~1-2 wei regardless of fee size, since the share-denominated ceiling self-corrects; a trivial buffer (redeem(maxRedeem() - 1)) is always sufficient there. No fund loss occurs in either case — both are integration-usability issues, not security vulnerabilities, but the safe-buffer size integrators need differs meaningfully between the two paths.
Recommendation	- For withdraw: re-query maxWithdraw(owner) as close to submission as possible, or size withdrawals with an explicit margin below the queried ceiling (not just -1), since the gap scales with elapsed-time/fee size. - For redeem: redeem(maxRedeem() - 1) is a sufficient safety buffer, since the residual effect is rounding-only. - Document both behaviors, distinguishing the asset-denominated (fee-proportional) staleness risk on withdraw from the share-denominated (rounding-only) risk on redeem.

Status	Fixed in commit 9a9164e. NatSpec now qualifies that withdraw(maxWithdraw()) can revert by a fee-proportional margin (asset-denominated ceiling), while documenting redeem(maxRedeem()) as the safe full-exit call (share-denominated, rounding-only residual):
	JavaScript
	`// maxWithdraw is asset-denominated: shrinks in real proportion to accrued fee`
	`// maxRedeem is share-denominated off balanceOf: does not shrink on accrual`